How-To Library

Informatica

Sizing Guidelines and Performance Tuning for Intelligent Streaming

© Copyright Informatica LLC 2017, 2021. Informatica and the Informatica logo are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html. Other company and product names may be trade names or trademarks of their respective owners.

Abstract

You can tune Intelligent Streaming for better performance. This article provides recommendations that you can use to tune hardware, Spark configuration, mapping configuration, and the Kafka cluster.

Supported Versions

- Informatica Intelligent Streaming 10.2

Table of Contents

Overview. . . . 2
Determine Your Hardware. . . . 2
Intelligent Streaming Deployment Types. . . . 3
Deployment Criteria. . . . 3
Deployment Type Comparison. . . . 3
Tune the Spark Engine. . . . 4
Tune Spark Parameters. . . . 4
Tune the Kafka Cluster. . . . 5
Tune the Mapping. . . . 6
Sizing Recommendations for Overhead Properties. . . . 6
Recommendations for Tuning the Kafka Cluster. . . . 6
Documentation Reference. . . . 7

Overview

Use Informatica Intelligent Streaming mappings to collect streaming data, build the business logic for the data, and push the logic to a Spark engine for processing. The Spark engine uses Spark Streaming to process data. The Spark engine reads the data, divides the data into micro batches, and publishes it.

Streaming mappings run continuously. When you create and run a streaming mapping, a Spark application is created on the Hadoop cluster which runs forever unless killed or cancelled through the Data Integration Service.

To optimize the performance of Intelligent Streaming and your system, perform the following tasks:

- Determine your hardware requirement.
- Tune the Spark engine.
- Tune the mapping.
- Tune the Kafka cluster.

Determine Your Hardware

To optimize the performance, acquire the right type of hardware for your Intelligent Streaming environment.

Consider the following points while determining your hardware:

- The criteria for determining a deployment is the number of messages processed and not the size of data in gigabytes.

- Choose the hardware such that batches do not get queued. Batches are queued when the batch processing time is greater than the batch interval time.
- Use the following approach to determine hardware capacity:
  - Implement a proof of concept (POC) to determine the throughput for each core.
  - Determine the peak load in terms of the anticipated number of the messages processed per second.
  - Divide the peak load by the throughput for each core to get the total number of cores required and the size of the disk, memory, and network appropriately.

Intelligent Streaming Deployment Types

Sizing and tuning recommendations vary based on the deployment type.

Based on certain deployment factors in the domain and Hadoop environments, Informatica categorizes Intelligent Streaming into the following types:

- Sandbox deployment
- Small deployment
- Medium deployment
- Large deployment

Deployment Criteria

The following criteria determine the Intelligent Streaming deployment type:

Number of messages processed

The total number of messages per second processed from a well-tuned Kafka cluster and written back to a well-tuned Kafka cluster.

Total cores

The total number of cores for each container.

Total memory

The maximum RAM available for each container.

Deployment Type Comparison

The following table compares Intelligent Streaming deployment types based on the standard values for each deployment factor:

Deployment	Messages Per Second	CPU	Memory
Sandbox	100,000	- Domain. 4 - Hadoop. 24	- Domain. 16 GB - Hadoop. 40 GB
Small	500,000	- Domain. 4 - Hadoop. 64	- Domain. 16 GB - Hadoop. 120 GB

Deployment	Messages Per Second	CPU	Memory
Medium	1 million	- Domain. 4 - Hadoop. 116	- Domain. 16 GB - Hadoop. 224 GB
Large	10 million	- Domain. 4 - Hadoop. 1056	- Domain. 16 GB - Hadoop. 2104 GB

Tune the Spark Engine

When you develop mappings in the Developer tool to run on the Spark engine, consider the following tuning recommendations and performance best practices:

Tune Spark parameters

To optimize Intelligent Streaming performance, tune Spark parameters in the `hadoopEnv.properties` file. To tune Spark parameters for specific mappings, configure the execution parameters of the Streaming mapping Run-time properties in the Developer tool. If you tune the parameters in the `hadoopEnv.properties` file, the configuration applies to all mappings that you create.

Tune the Kafka cluster

To optimize the performance of the Kafka cluster, configure the number of the nodes and brokers per node in the Kafka cluster.

Tune Spark Parameters

Tune the Spark parameters in the hadoopEnv.properties file.

The `hadoopEnv.properties` file is located in the following directory:

```
<Informatica installation directory>/services/shared/hadoop/<Hadoop distribution name>/infaConf
```

If you tune the parameters in the `hadoopEnv.properties file`, the configuration applies to all mappings that you create.

You can configure the following parameters based on the input data rate, mapping complexity, and concurrency of mappings:

spark.executor.cores

The number of cores to use on each executor.

Recommended value: Specify 3 to 4 cores for each executor. Specifying a higher number of cores might lead to performance degradation.

spark.executor.memory

The amount of memory to use for each executor process.

Recommended value: Specify a value of 8 GB.

spark.driver.memory

The amount of memory to use for the driver process.

Recommended value: Specify a value of 8 GB.

spark.driver.cores

The number of cores to use for each driver process.

Recommended value: Specify 8 cores.

spark.executor.instances

The total number of executors to be started. This number depends on number of machines in the cluster, memory allocated, and cores per machine.

Configure the number of executor instances based on the following deployment types:

- Sandbox deployment. 4
- Small deployment. 14
- Medium deployment. 27
- Large deployment. 262

spark.sql.shuffle.partitions

The total number of partitions used for a SQL shuffle operation.

Recommended value: Specify a value that equals the total number of executor cores if total executor cores allocated is less than 200. Maximum value is 200.

Configure the partitions based on the following deployment types:

- Sandbox deployment. 16
- Small deployment. 56
- Medium deployment. 108
- Large deployment. 200

spark.kryo.registrationRequired

Indicates whether registration with Kryo is required.

Recommended value: True

spark.kryo.classesToRegister

The comma-separated list of custom class names to register with Kryo if you use Kyro serialization.

Specify the following value for all deployment types:

```
org.apache.spark.sql.catalyst.expressions.GenericRow,[Ljava.lang.Object;,
org.apache.spark.sql.catalyst.expressions.GenericRowWithSchema,
org.apache.spark.sql.types.StructType,[Lorg.apache.spark.sql.types.StructField;,
org.apache.spark.sql.types.StructField,
org.apache.spark.sql.types.StringType$,
org.apache.spark.sql.types.Metadata,
scala.collection.immutable.Map$EmptyMap$
[Lorg.apache.spark.sql.catalyst.InternalRow;,
scala.reflect.ClassTag$$anon$1,java.lang.Class
```

Tune the Kafka Cluster

The following table lists the Kafka cluster properties that you can tune based on the deployment type :

Property	Sandbox	Small	Medium	Large
Number of Kafka brokers for each node	1	2	4	8
Number of Kafka nodes	1	2	4	8

Note: Kafka brokers can be consumers or producers.

Tune the Mapping

To tune the mapping, use the sizing recommendations based on mapping complexity.

Mapping complexity is defined by the number of sources, targets, and transformations in the mapping. Transformations can be CPU bound (Expression transformation), memory bound (Lookup transformation), or disk bound.

Mappings can be grouped into following categories of complexity:

Complexity	Sources	Transformations	Targets
Simple	3	6 (3 memory bound and 3 CPU bound)	1
Standard	7	10 (8 memory bound and 2 CPU bound)	1
Complex	6	13 (8 memory bound and 3 CPU bound)	1

Sizing Recommendations for Overhead Properties

The following table lists the overhead requirements based on mapping complexity:

Mapping Complexity	VCPUs Overhead	Memory Overhead
Simple	33%	33%
Standard	120%	120%
Complex	150%	150%

Recommendations for Tuning the Kafka Cluster

Consider the following recommendations to tune the Kafka cluster:

- Configure the Kafka cluster so that Intelligent Streaming can produce and consume messages at the needed message ingestion rate.
- To increase the rate of message consumption in Intelligent Streaming, increase the number of Kafka brokers in the Kafka cluster and in the Kafka connection.
- Increase the number of partitions on the Kafka topic. Ideally, the number of partitions can be equal to the number of CPU cores allocated to the executors. For example, if you set `spark.executor.instances` to 6 and `spark.executor.cores` to 3, there are 18 cores allocated. Then set the number of Kafka partitions to 18, so that there are 18 parallel tasks to read from the Kafka Source.
  For example, you can use the following command to specify the number of partitions:

  ```
  ./ kafka-topics.sh --create --zookeeper
  zookeeper_host_name1:zookeeper_port_number ,zookeeper_host_name2:zookeeper_port_number,zoo
  keeper_host_name3:zookeeper_port_number --replication-factor 1 --partitions 18 --topic
  NewOSConfigSrc
  ```
- Ensure that the Kafka producer is publishing messages to every partition in a load balanced manner.
- Reduce the number of network hops between Intelligent Streaming and the Kafka cluster. Ideally the Kafka broker must be on the same machine as the data node or the Kafka cluster can run on its own machines with a zero latency network.

- Configure the `batch.size` and `linger.ms` properties to increase throughput. For each partition, the producer maintains buffers of unsent records. The `batch.size` property specifies the size of the buffer. To accumulate as many messages as possible in the buffer, configure a high value for the `batch.size` property.
  By default, the buffer sends messages immediately. To increase the time that the producer waits before sending messages in a batch, set the `linger.ms` property to 5 milliseconds.
- Kafka scalability depends on disk and network performance. The test setup included 12 disks per node on a 10 GBPS network with an open file limit of 65000.

Documentation Reference

See the following performance-related How-To Library articles for Informatica big data products:

- Tuning the Hardware and Hadoop Clusters for Informatica Big Data Products. Provides tuning recommendations for the hardware and the Hadoop cluster for better performance of Informatica big data products.
- Performance Tuning and Sizing Guidelines for Big Data Management 10.2. Provides sizing recommendations for the Hadoop cluster and the Informatica domain, tuning recommendations for various Big Data Management components, best practices to design efficient mappings, troubleshooting tips, and case studies.

Authors

Vidya Vasudevan

Shahbaz Hussain